FLYRANK.AI · AI BACKEND ENGINEERING · PROJECT BRIEF

Build your first CRUD API

Build a small API that manages a to-do list — create, read, update and delete tasks — test it in Swagger UI, and publish it to GitHub.

~6–8 h across 6 stages | Stretch +2 h | Bonus AI stage +1 h | JavaScript or Python | $0 · no credit card

COMPANION DECK	YOU WILL PRACTICE	DELIVERABLE
Backend 101 — how the web works	**HTTP · CRUD · status codes · Swagger UI · Git & GitHub**	**Public GitHub repo, ≥6 commits, README**

How to read this document: new words are shown in **bold** the first time they appear — every one of them is explained in the Glossary at the end. If a sentence confuses you, check the glossary first: it's probably one word, not the whole idea. Work the stages **in order**; each ends with a checkpoint that proves it works.

Contents

1. Goal & purpose 1
2. The big idea in 60 seconds 2
3. Tools — pick one lane 2
4. The project — six stages 2
5. Bonus stage — the AI rematch 6
6. Requirements & stretch goals 6
7. Done means 7
8. Curated resources 8
9. Glossary 12

1 · Goal & purpose

Goal: build a small **API** that manages a to-do list: you can **create** tasks, **read** them, **update** them, and **delete** them — the four **CRUD** operations. You will see and test your API in a visual page called **Swagger UI**, and publish everything to **GitHub**.

The companion deck shows the request → response loop from the outside. Here you build the server side of it yourself. CRUD is the heartbeat of almost every backend in the world — a social network CRUDs posts, a shop CRUDs orders, FlyRank CRUDs SEO reports. Once you've built CRUD once, every backend you ever meet will feel familiar.

Two habits start here: **your data lives only in memory** (no database yet — persistence is the natural next step, and losing your data on restart is a lesson, not a bug), and **everything is published through GitHub** (that's how all engineering work here is shared).

Beginners usually overthink this. The whole thing is **under 100 lines of code**, built in small stages. You never write more than ~15 lines before you can test again.

2 · The big idea in 60 seconds

Your API is a **server** — a program that waits for **requests** and sends back **responses**. It offers several **endpoints**. An endpoint is one "door" into your server, defined by two things:

1. a **path** — *where* the door is, like `/tasks` or `/tasks/3`
2. an **HTTP method** — *what kind* of knock it answers to: `GET` (give me), `POST` (create this), `PUT` (replace this), `DELETE` (remove this)

So `GET /tasks` ("give me all tasks") and `POST /tasks` ("create a task") are **two different endpoints**, even though the path is the same. The four CRUD operations map onto the methods like this:

CRUD operation	HTTP method	Example endpoint	Meaning
Create	`POST`	`POST /tasks`	Add a new task
Read	`GET`	`GET /tasks` · `GET /tasks/3`	List all tasks / get task 3
Update	`PUT`	`PUT /tasks/3`	Change task 3
Delete	`DELETE`	`DELETE /tasks/3`	Remove task 3

That table is the project. Everything below just builds it, one row at a time.

3 · Tools — pick ONE lane

Both lanes build exactly the same API. Pick the language you want to stick with; don't switch mid-project.

	JavaScript lane	Python lane
Language	Node.js (free, nodejs.org)	Python 3.10+ (free, python.org)
Framework	Express — Hello world	FastAPI — First steps
Swagger UI	Add with swagger-ui-express (Stage 5)	**Built in** at `/docs` — zero setup
Testing your API	**curl** + browser + Hoppscotch (all free)	same
Publishing	Git + a free GitHub account	same

> Not sure? If you're more comfortable with JavaScript, take the JavaScript lane. If Python feels friendlier, take the Python lane — you'll get Swagger for free, which is a nice reward.

4 · The project — six stages (+ one bonus)

Work stage by stage, **in order**. Each stage ends with a **checkpoint**: a command you run to prove it works. **Commit to Git after every stage** (that's your ≥6 commits, honestly earned). If you only finish Stage 3, push it anyway — a working half is worth more than a broken whole.

0 Hello, server

~30 min

The scene: before a restaurant serves food, the doors have to open.

1. Install your lane's tools (Node or Python — see Resources §6).
2. Follow your framework's official hello-world page (linked in the table above) to start a server on **localhost** — Express on **port** 3000, FastAPI on port 8000.
3. Visit it in your browser. You should see your hello message.

> `CHECKPOINT` — `curl -i http://localhost:3000/` (or `:8000/`) returns **status code** `200` and your message.

Commit: `Stage 0: hello server`

1 Your first real endpoint

~45 min

Every API needs a front door that says what it is.

1. Add the endpoint `GET /` returning **JSON** that describes your API:

```
{ "name": "Task API", "version": "1.0", "endpoints": ["/tasks"] }
```

2. Add `GET /health` returning `{ "status": "ok" }`. Real companies use exactly this endpoint to check a server is alive — you've just built your first professional habit.

> `CHECKPOINT` — both URLs return JSON in the browser and via curl.

Commit: `Stage 1: root and health endpoints`

2 Read: list and single task

~1 h

Now the shelves. Your "database" is just a list in your code.

1. Near the top of your file, create an **in-memory** list of task objects, pre-filled with 3 example tasks. Each task has: `id` (number), `title` (text), `done` (true/false).
2. Add `GET /tasks` — returns the whole list.
3. Add `GET /tasks/:id` (Express) / `GET /tasks/{id}` (FastAPI) — returns **one** task. The `id` part is a **path parameter**: a piece of the URL that changes.
4. If no task has that id, return status **404** with a JSON error: `{ "error": "Task 99 not found" }`. Never return an empty `200` for something that doesn't exist — status codes are how machines read your answers.

> `CHECKPOINT` — `curl -i http://localhost:3000/tasks/1` → `200` + one task · `curl -i http://localhost:3000/tasks/99` → `404` + error JSON.

Commit: `Stage 2: read endpoints with 404`

3 Create: POST a new task

~1 h

A customer walks in with a new order.

1. Add `POST /tasks`. The client sends the new task as JSON in the **request body**:

```
{ "title": "Implement core CRUD endpoints with Pydantic validation" }
```

2. Your server: gives it the next free `id`, sets `done` to `false`, adds it to the list, and returns the created task with status **201** ("Created" — the polite way to say "done, here's your receipt").
3. **Validate** the input: if `title` is missing or empty, return **400** ("Bad Request") with a JSON error saying what's wrong. This is your first business rule — **the server never trusts the client**.

> `CHECKPOINT` —
>
> ```
> curl -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Implement core CRUD endpoints with Pydantic validation"}'
> ```
>
> returns `201` + the new task, and a second `GET /tasks` shows it in the list. Posting `{}` returns `400`.

Commit: `Stage 3: create with validation`

4 Update & Delete

~1 h

Orders change, orders get cancelled.

1. Add `PUT /tasks/:id` — replaces a task's `title` and/or `done` with what's in the request body. Returns the updated task. Unknown id → `404`. Empty/invalid body → `400`.
2. Add `DELETE /tasks/:id` — removes the task. Return status **204** ("No Content" — success, nothing to say) with an empty body. Unknown id → `404`.
3. Stop and notice: **you have built a complete CRUD API.** Every backend you'll ever work on is this, wearing more clothes.

> `CHECKPOINT` — create a task, update it, mark it done, delete it, and confirm with `GET /tasks` — all via curl, all with the right status codes (`201` , `200` , `204` , `404`).

Commit: `Stage 4: full CRUD`

5 See it: Swagger UI

~1–1.5 h

So far you've imagined your API. Now look at it.

Swagger UI is a web page that reads a description of your API (an **OpenAPI** file) and turns it into interactive documentation: every endpoint listed, with a **Try it out** button that sends real requests — curl with a friendly face.

- **Python lane:** open `http://localhost:8000/docs`. It's already there — FastAPI generates it from your code. Add a one-line description to each endpoint (see First steps) and watch the page improve.
- **JavaScript lane:** install swagger-ui-express, write a small `openapi.json` describing your five task endpoints (the package README shows the wiring; OpenAPI basic structure explains the file). Serve it at `/docs`. Describing endpoints you already built teaches you more than building them did.

Then, in Swagger UI, without curl: create a task, list tasks, update it, delete it.

> `CHECKPOINT` — `/docs` shows all your endpoints; "Try it out" works for the full CRUD cycle. Take a screenshot for your README.

Commit: `Stage 5: Swagger UI`

6 Publish to GitHub

~1 h

Your work only counts when someone else can run it.

1. Create a **public** GitHub **repo** and push your code (your ≥6 stage commits come with it).
2. Write a **README** with: what this is, how to install & run it (one documented command), a table of all endpoints, one pasted `curl -i` output, and your Swagger screenshot.

> New to Git? The basics are all you need here: `init → add → commit → push` — see Resources §9. Branches, pull requests and team code review are the natural next step once this is shipped.

> `CHECKPOINT` — a stranger with your README could run your API in under 5 minutes.

Commit: `Stage 6: publish and docs` — then push everything.

★ Make it yours — optional extras

optional · as many or as few as you like

No database yet — so let's have fun with what memory can do.

None of these are required. Pick whatever sounds fun (creative alternatives welcome):

- **Filtering with query parameters:** `GET /tasks?done=true` returns only finished tasks. A **query parameter** is the part after `?` — filters, not addresses.
- **Search:** `GET /tasks?search=milk` returns tasks whose title contains the word.
- **A stats endpoint:** `GET /stats` → `{ "total": 7, "done": 3, "open": 4 }` — your first taste of the server computing something instead of just storing it.
- **Seed & reset:** `POST /reset` restores the 3 example tasks. Handy for demos — and for the next point.
- **The persistence experiment:** create a few tasks, restart your server, `GET /tasks`. Write two sentences in your README about what happened and why. This observation is the entire reason databases exist.

Commit (if you build any): `Extras: <what you added>`

5 · Bonus stage — the AI rematch

7 The AI rematch

~1 h · optional — and the most fun

You built this API by hand, line by line. Now hire the fastest junior developer on Earth — and review their work.

You did Stages 0–6 by hand for a reason: you now know exactly what "correct" looks like. That knowledge is what turns this stage from a magic show into a **code review**.

1. **Write the prompt yourself — this is the real exercise.** Without copying text from this document, write your own **prompt** asking an AI assistant (Claude, ChatGPT, Gemini — any) to build the same API. From memory, try to specify everything that matters: language and framework, the five endpoints, status codes, validation rules, in-memory storage, Swagger UI. Describing a system precisely is a core backend skill — it is the foundation of spec-first development.
2. **Generate in quarantine.** Put the AI's code in a separate folder (`ai-version/`) or a branch. Your Stages 0–6 code stays untouched — that is your hand-built implementation, and it must stay hand-built.
3. **Run it.** Does it start on the first try? Fire your Stage 4 checkpoint curls at it. Which pass? Which fail?
4. **Diff it.** Compare the AI's code with yours side by side (`git diff --no-index your-file ai-file` works on any two files). Then answer three questions in a short **"AI vs me"** section of your README:
   - What did the AI do *better* — and do you understand its version well enough to explain it?
   - What did it get *wrong* or quietly ignore from your prompt? (A missing `400` ? A wrong status code? A database you never asked for?)
   - What did your *prompt* forget to specify — and what did the AI silently decide for you?
5. **One rematch.** Improve your prompt with what you learned, regenerate, and note in one sentence what changed.

> The lesson hiding in this stage: an AI's output is exactly as good as your specification — and you could only judge it because you had built the thing yourself first. **Both halves of that sentence are your career from now on.**

> `CHECKPOINT` — your README has an "AI vs me" section containing your full prompt and at least three concrete differences you found.

Commit: `Stage 7: AI vs me` (AI code stays in its own folder/branch).

6 · Requirements

Done = every box ticked. Each one is checkable in under a minute.

- [ ] Server starts with **one documented command** on localhost.
- [ ] `GET /tasks`, `GET /tasks/:id`, `POST /tasks`, `PUT /tasks/:id`, `DELETE /tasks/:id` all work — full CRUD on an **in-memory** list (no database, no files).
- [ ] Correct status codes: `200` reads, `201` create, `204` delete, `400` invalid body, `404` unknown id — each error with a JSON `error` message.
- [ ] `POST` and `PUT` **validate** input (missing/empty `title` → `400`).
- [ ] **Swagger UI at** `/docs` lists every endpoint, and the full CRUD cycle works via "Try it out".
- [ ] Public **GitHub repo**, ≥6 meaningful commits (one per stage), README with run instructions, endpoint table, one `curl -i` output, and the Swagger screenshot.

Stretch (optional)

- [ ] **Express lane:** generate your OpenAPI spec from code comments with `swagger-jsdoc` (LogRocket tutorial) instead of a hand-written file.
- [ ] Add pagination: `GET /tasks?limit=2&offset=2` — and explain in the README why real APIs never return "everything".
- [ ] **Stage 7 — the AI rematch** (above): prompt an AI to build the same API, run it, diff it, write your "AI vs me" section.

7 · Done means

- The full CRUD cycle works **twice**: once via `curl -i` (right status codes visible), once via Swagger UI "Try it out".
- Your repo is public, the README works on a clean machine, and `git log` shows one honest commit per stage.

8 · Curated resources

Don't read everything. Each section says *when* you need it. All resources are free, no credit card. If a link dies, search the title — these are all well-known materials.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive**

§1 · How the web works — before you start

	Resource	Format	Why it's useful
●	MDN — How the web works	Article, ~10 min	The single best plain-language explanation of what happens when you open a website. Read this **before the companion deck** — the live demos will make twice as much sense.
●	MDN — How does the Internet work?	Article, ~10 min	One level below the web: cables, networks, and how computers find each other. Explains the "internet vs web" difference everyone quietly wonders about.
●	What happens when you type google.com and press Enter	GitHub repo	The famous interview question, answered in extreme depth. Skim it once now, come back after you've shipped a few backends and be amazed how much you understand.

§2 · HTTP — the language of the loop

	Resource	Format	Why it's useful
●	MDN — Overview of HTTP	Article, ~15 min	The reference explanation of requests, responses, methods and headers. This is the page the deck's "loop" slide is built on.
●	HTTP Crash Course — Traversy Media	Video, ~40 min	The best beginner video on HTTP: request/response cycle, methods, status codes, headers — with live demos very similar to the deck's. Great to re-watch after going through it.
●	MDN — HTTP response status codes	Reference	The full list of status codes. Bookmark it — you'll use it when deciding between 200, 201, 400 and 404.

http.cat	Fun reference	Every status code as a cat photo. Sounds silly, works scarily well as a memory aid. 404 = cat not found.
Launch School — Introductory HTTP (free book)	E-book, ~2 h	One calm, complete, beginner-paced walkthrough of HTTP from zero. Read over a weekend.
freeCodeCamp — An introduction to HTTP	Article, ~20 min	A good second pass on the same material in different words — helpful when MDN's phrasing didn't click.

§3 · APIs, REST & CRUD — the vocabulary of the project

Resource	Format	Why it's useful
MDN — HTTP request methods	Reference	GET, POST, PUT, PATCH, DELETE — exactly the five verbs your CRUD API uses. One page, keep it open while building.
Understanding RESTful API CRUD operations — Treblle	Article, ~15 min	Maps CRUD to HTTP methods with clear examples — the exact mental model this project asks you to build.
How to design a RESTful API with CRUD — Zuplo	Article, ~20 min	Goes one step further: URL naming, status code choices, and common design mistakes. Read after your API works, then improve it.
restfulapi.net	Site	The "textbook" on REST principles. Use it to answer "is my URL design right?" questions.
restful-api.dev	Practice API	A real public API that supports full CRUD. Practice sending GET/POST/PUT/DELETE against someone else's server before building your own.
JSONPlaceholder	Practice API	The fake API from the deck's demo. Free, no signup — perfect for curl practice.

§4 · JSON — the data format everything speaks

Resource	Format	Why it's useful
MDN — Working with JSON	Article, ~15 min	Every request and response in this project carries JSON. This explains the format in 15 minutes; you'll never need another JSON tutorial.

§5 · Tools — calling APIs like a developer

Resource	Format	Why it's useful
Hoppscotch	Free web tool	A friendly, browser-based way to send requests to your API — no install, no account. Great when curl feels intimidating; use both.
curl (already installed) — Everything curl: the basics	Book chapter	curl is the professional's tool and the deck uses it. You only need the basics chapter — how to GET, POST, and see headers with `-i`.
Postman	Free app / web	The industry-standard API client: save your requests into a collection, re-run the whole CRUD cycle with one click, and share it. You'll meet Postman in almost every backend team — the free tier covers everything this project needs.

§6 · Framework quickstarts — pick your lane

Resource	Format	Why it's useful

Express — Hello world	Official docs	The exact starting point for the JavaScript lane. ~10 lines, copy and understand each one.
Express — Basic routing + Routing guide	Official docs	How endpoints ("routes") are defined in Express, including path parameters like `/tasks/:id` — the heart of the project.
FastAPI — First steps	Official docs	The exact starting point for the Python lane. FastAPI's tutorial is famously beginner-friendly — follow it step by step.
FastAPI — Path parameters + Request body	Official docs	The two pages that cover everything the CRUD project needs: reading `/tasks/{id}` and receiving JSON in a POST.

§6b · Build-along tutorials — watch the whole project get coded

These walk through building a CRUD API end to end — very close to what you're building. Watching or typing along is allowed and encouraged; just make the final API yours (task fields, your validation, your extras).

Resource	Format	Why it's useful
JS lane: freeCodeCamp — How to create a CRUD API with Node & Express	Article, ~45 min	Builds a full CRUD API on an in-memory list — the same shape as this project (books instead of tasks). Perfect worked example when you're stuck in Stages 2–4.
JS lane: Build a REST API with Node JS and Express (YouTube)	Video, ~1 h	Watch every endpoint of a CRUD API being written and tested live: routes, path parameters, POST bodies, status codes. Watch a stage ahead of where you are, then build it yourself.
PY lane: freeCodeCamp — FastAPI Course for Beginners (YouTube)	Video, ~1 h	Path parameters, query parameters, request bodies, POST/PUT/DELETE — the exact toolkit of Stages 2–4, demonstrated live in FastAPI.
PY lane: Real Python — Using FastAPI to Build Python Web APIs	Article, ~25 min	A calm second pass over the official tutorial's ideas: type hints, validation, and why `/docs` appears for free. Read after your first endpoints work.
JS lane: GeeksforGeeks — REST API CRUD operations using Express	Article, ~20 min	A compact written reference of all four CRUD routes in Express on in-memory data — handy to compare against your own code, stage by stage.

§7 · Swagger / OpenAPI — see your API

Resource	Format	Why it's useful
Swagger UI — what it is	Overview	Swagger UI turns your API into an interactive web page where you can click "Try it out" instead of typing curl commands. Stage 5 uses it.
FastAPI lane: interactive docs are built in	Official docs	If you chose Python: you get Swagger UI for free at `http://localhost:8000/docs`. Zero setup — this page shows what you'll see.
Express lane: swagger-ui-express (npm)	Package docs	If you chose JavaScript: this package serves Swagger UI at `/docs` from a small spec file you write. The README's example is all you need.
Documenting your Express API with Swagger — LogRocket	Tutorial, ~30 min	A full walkthrough of swagger-ui-express + swagger-jsdoc (spec generated from code comments). Use it for the stretch goal.
OpenAPI 3.0 — basic structure	Official docs	What the `openapi.json/yaml` file actually contains. Read this when you edit your spec and wonder what `paths`, `parameters` and `responses` mean.

§8 · The bigger map

	Resource	Format	Why it's useful
●	roadmap.sh — Backend Developer	Interactive roadmap	The community-standard map of everything backend. Use it to see where this project fits in the wider world.
●	roadmap.sh — Backend projects	Project list	Extra practice projects, easiest first. If this project felt too easy, pick one of these.

§9 · Git & GitHub — publish your work

> **Heads-up:** for this project you only need the survival basics: `init → add → commit → push`. Learn those below; branches, pull requests and team workflows can come later.

	Resource	Format	Why it's useful
●	GitHub Docs — Hello World	Official guide, ~15 min	GitHub's own quickstart: create a repository, make a change, commit it. The fastest path from "no account" to "my code is online".
●	freeCodeCamp — Git & GitHub Crash Course for Beginners (YouTube)	Video, ~1 h 20	Why version control matters, then hands-on: add, commit, status, log, branches, push and pull. Everything Stage 6 asks of you, shown on screen.
●	freeCodeCamp — Git & GitHub beginner-friendly handbook	Article / reference	A written reference to keep open while you work — look up any command the moment it confuses you.
●	Learn Git Branching	Interactive game	Practice commits and branches visually, level by level. Weirdly addictive — and the perfect warm-up for team workflows.

Suggested path for a complete beginner

1. **Before the deck:** §1 (both green MDN articles) — ~20 min.
2. **After the deck:** Traversy's HTTP crash course video + MDN JSON — ~1 h.
3. **While building:** keep §3, §6, §6b and §7 open as references; use http.cat when a status code confuses you.
4. **Stuck or curious:** the yellow rows. Everything orange is optional forever.

Critical resources per stage

Stage	Reach for
Stage 0–1	Your lane's official quickstart (§6) + a basic terminal guide.
Stage 2–4	MDN HTTP methods · CRUD ↔ REST explained (Treblle) · the build-along tutorials in §6b · http.cat when a status code confuses you.
Stage 5	§7 (Swagger/OpenAPI) — pick your lane's row.
Stage 6	§9 (Git & GitHub) — init → add → commit → push.
Stage 7 (bonus)	Any AI assistant with a free tier; `git diff --no-index` for comparing files.

9 · Glossary

Plain-language definitions of every bold word above. No definition depends on another — read them in any order.

Word	What it means
API	A set of doors a program offers so other programs can talk to it. Your to-do API lets any client create and read tasks by sending requests.

Endpoint	One specific door: a path plus a method. `GET /tasks` and `POST /tasks` are two different endpoints.
Path	The part of the URL after the domain: in `http://localhost:3000/tasks/3`, the path is `/tasks/3`.
HTTP method	The type of action a request asks for: GET (give me), POST (create), PUT (replace), PATCH (change part), DELETE (remove). Also called an HTTP "verb".
CRUD	Create, Read, Update, Delete — the four things almost every app does with its data.
Server	A program that runs and waits for requests, then answers them. (The word also means the machine it runs on.)
Client	Whatever sends the request: a browser, curl, Swagger UI, a mobile app.
Request	A message from client to server: "method + path + maybe a body".
Response	The server's answer: "status code + headers + usually a body".
Request body	The data a client sends inside a request — e.g. the JSON of a new task in a POST.
Status code	A 3-digit number in every response saying how it went. 2xx = worked (200 OK, 201 Created, 204 No Content) · 4xx = the client goofed (400 Bad Request, 404 Not Found) · 5xx = the server goofed.
JSON	The text format APIs use for data: `{ "title": "Buy milk", "done": false }`. Easy for humans to read, easy for machines to parse.
localhost	Your own computer's address. `http://localhost:3000` = "the server running on this machine, port 3000". Nobody else can see it.
Port	A numbered doorway on a machine, so many programs can listen at once. Express commonly uses 3000, FastAPI 8000.
In-memory	Data kept in your program's variables (a list in the code). Fast and simple — but gone when the program stops. Databases exist to fix exactly this.
Path parameter	A changing piece of the path: the `3` in `/tasks/3`. Written `:id` (Express) or `{id}` (FastAPI) in your code.
Query parameter	Extra options after `?` in the URL: `/tasks?done=true&search=milk`. Used for filtering and searching, not for identifying a resource.
Validate / validation	Checking incoming data before trusting it (is `title` there? is it text?). The server never assumes the client behaved.
Framework	A box of pre-solved problems (routing, JSON parsing, responses) so you only write your own logic. Express and FastAPI are frameworks.
Swagger UI	A web page that displays your API as interactive documentation, with a "Try it out" button that sends real requests.
OpenAPI	The standard file format (JSON or YAML) that describes an API — its endpoints, inputs and responses. Swagger UI reads this file. FastAPI writes it for you; in Express you write it yourself.
curl	A terminal program that sends HTTP requests. `curl -i URL` shows the status code and headers too.
Git / GitHub / repo / commit	Git tracks versions of your code in a **repository**; a **commit** is one saved step with a message; GitHub hosts your repo online so others can see it.
README	The front page of a repo: what this is, how to run it, how to use it.

Route	Another word for an endpoint definition in your code — frameworks call the code that answers an endpoint a "route" or "route handler".
Prompt	The instructions you give an AI assistant. In Stage 7 your prompt is a mini-specification: the more precisely it names endpoints, status codes and rules, the closer the output lands to your API.
Diff	A line-by-line comparison of two versions of code, showing what was added, removed or changed. `git diff` produces one; reading diffs is how professionals review each other's work.

FlyRank.ai · AI Backend Engineering · Project Brief — Build your first CRUD API. All linked resources are free with no credit card required.